General - RPS Data

NIIT TRAINING DIVISION | LTM - iRUN Full Stack Engineering

TRAINING LAB GUIDE

LAB EXERCISE - 2

Topics, Partitions & Producer Configuration — Publish Messages

Publish Unkeyed Messages · Publish Keyed Messages · Change Partition Count Safely · Tune Producer Settings

Module Series	Apache Kafka Operations & Streaming Platforms — Day 4
Lab Type	Guided Hands-On SOP — Kafka Producers & Partitioning
Duration	48 minutes (5 min setup + 25 min tasks + 10 min validation + 8 min debrief)
Difficulty	⭐⭐ `Beginner – Fully Guided with Conceptual Depth`
Platform	Linux lab VM with Docker Engine + Compose v2; Apache Kafka 4.3.1 (apache/kafka image), 3-node KRaft cluster
Document Version	v1.0 | IT Training Division | 2026
Learner Name	_____
Date & Batch	_____
Prerequisite Lab Completed	☐ Day 4 Lab 1 (Kafka Architecture — KRaft Cluster and Topics) — signed off; cluster running
Instructor Sign-Off	_____

1. LAB OVERVIEW

1.1 Why This Lab Exists

Where a message lands in Kafka is decided by the producer, not the broker. Messages with the same key always go to the same partition — which is what preserves per-key ordering — while messages without a key are spread across partitions in batches. The partition count, the choice of key and the producer's acks, batching and compression settings decide ordering, balance, durability and throughput. In this lab you publish unkeyed and keyed messages, predict and verify exactly where each key lands, see what happens to key placement when partitions are added, and measure how producer configuration changes throughput and latency.

NIIT iRUN Alignment

INCIDENT OWNERSHIP: The order-tracking team says events for the same order arrive out of sequence. You own proving where each key is written and whether a partition change broke ordering.

TROUBLESHOOTING EXCELLENCE: You will predict key placement with Kafka's own hash function and confirm it with kafka-get-offsets — evidence, not assumptions.

OPERATIONAL READINESS: Producer settings (acks, linger.ms, batch.size, compression) are tuned by measurement with kafka-producer-perf-test, not copied from a blog.

1.2 Lab Scenario

Incident Briefing — Out-of-Order Store Events

TIME: 10:30 | PRIORITY: HIGH | ASSIGNED TO: You

REQUEST RECEIVED: "Store events must be processed in order per store. Last week someone added partitions to the orders topic and ordering problems started. Also, the producers are slow during peak hours. Investigate both."

CONTEXT: The orders topic (6 partitions) exists from Lab 1. data/keyed_orders.txt holds 12 orders from 6 stores, keyed by store ID.

YOUR MISSION: Publish unkeyed and keyed messages, verify the exact partition of each store, reproduce the effect of adding partitions, and compare producer configurations. You have 25 minutes for the core tasks.

Figure: Lab 2 — Starting Environment: Cluster Running, Empty orders/payments/audit-log Topics

1.3 Conceptual Foundation

The producer's partitioner chooses a partition for every record. Its rule depends on whether the record has a key.

Figure: Producer Partitioning — Keyed: hash(key) mod partitions | Unkeyed: sticky batches across partitions

General - RPS Data

Record Type	Partition Choice	Ordering Guarantee	Typical Use
Keyed	murmur2(key) → positive → modulo number of partitions	All records with the same key in order	Orders per store, events per customer
Unkeyed	Sticky partitioner: fills a batch for one partition, then moves on	No cross-record ordering	Logs, clickstream, metrics
Explicit partition	Producer names the partition	Within that partition	Special cases only
Partition count change	Same key may hash to a DIFFERENT partition afterwards	Broken across the change	Plan partitions up front

Figure: Producer Settings That Matter — acks (durability) · linger.ms + batch.size (batching) · compression.type (bytes on the wire)

1.4 Learning Objectives

- Publish messages with the console producer, with and without keys
- Explain the default partitioner and predict the partition of a key
- Verify message placement per partition with kafka-get-offsets and the console consumer
- Explain why increasing partitions moves keys and breaks per-key ordering
- Measure the effect of acks, batching and compression with kafka-producer-perf-test
- Verify each step using documented evidence: per-partition offsets, printed keys and perf-test results

1.5 What You Will NOT Do

Hard Stops — Absolute Prohibitions for This Lab

✗ NEVER increase partitions on a production topic without checking who relies on per-key ordering

✗ NEVER run perf tests against orders, payments or audit-log — use the perf-test topic only

✗ NEVER run perf tests larger than this guide specifies — the VM shares resources with other learners

✗ NEVER edit the data files in data/ — the expected results depend on their exact contents

✗ NEVER proceed past Task 2 unless your per-partition offsets match the expected values exactly

2. PREREQUISITES & SETUP

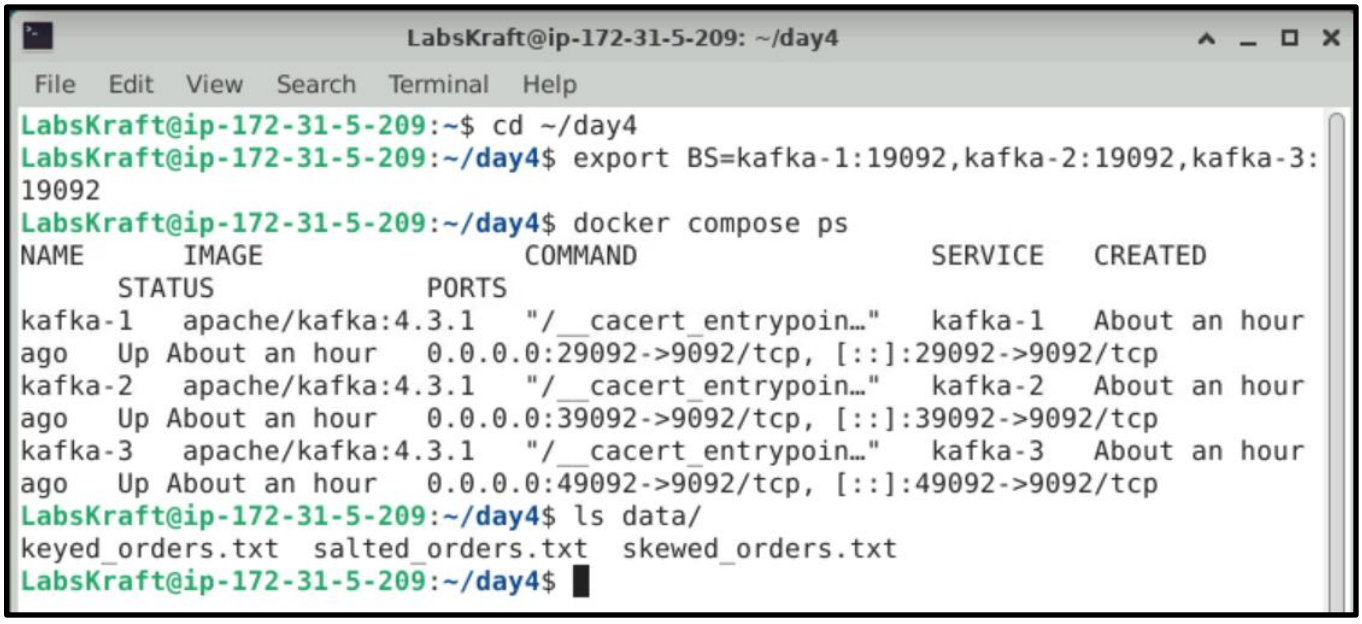
2.1 Knowledge Prerequisites

Prerequisite	Where Covered	Verified?
Kafka components; topics and partitions	Day 4 Lab 1	☐
Partition strategy and message keys	Day 4 Theory — Session 2	☐
Producer configuration: acks, batching, compression	Day 4 Theory — Session 2	☐
Hash functions and modulo (concept)	Foundation Module 2	☐
Using two terminal windows	Foundation Module 1	☐

2.2 Tools & Access Required

⌨ Pre-Lab — Verify cluster and files

```
cd ~/day4
export BS=kafka-1:19092,kafka-2:19092,kafka-3:19092
docker compose ps
ls data/
# Expected: 3 containers Up; keyed_orders.txt  salted_orders.txt  skewed_orders.txt
```



2.3 Pre-Lab Environment State Verification

Run all checks below and confirm the output matches expectations before starting the timed tasks:

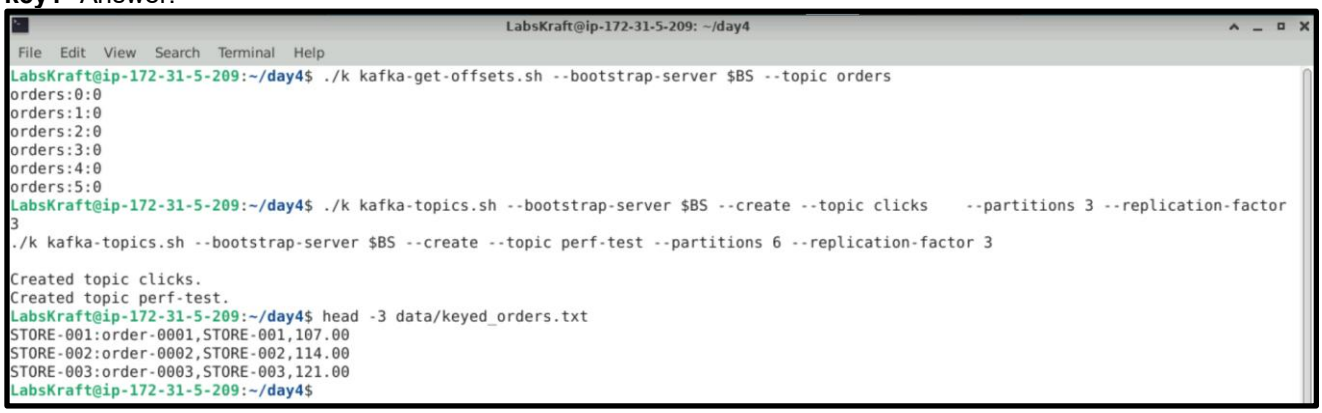
Pre-Lab — Verify starting state

```
# 1. orders is empty: every partition's end offset is 0
./k kafka-get-offsets.sh --bootstrap-server $BS --topic orders
# Expected: orders:0:0  orders:1:0 ... orders:5:0

# 2. Create the two topics this lab needs
./k kafka-topics.sh --bootstrap-server $BS --create --topic clicks    --partitions 3 --replication-factor 3
./k kafka-topics.sh --bootstrap-server $BS --create --topic perf-test --partitions 6 --replication-factor 3

# 3. Look at the keyed data file: key and value are separated by ':'
head -3 data/keyed_orders.txt
```

PRE-LAB: Record the end offsets of orders and the first line of keyed_orders.txt. Which part is the key? Answer:



3. CONCEPT DEEP-DIVE BEFORE STARTING

3.1 How a Key Becomes a Partition

For a keyed record the default partitioner computes murmur2(key bytes), makes it positive, and takes the remainder after dividing by the number of partitions. The result is deterministic: the same key and the same partition count always give the same partition. It is NOT a round-robin: two different keys can collide on one partition while another partition receives nothing.

Why This Matters Operationally

Because placement is deterministic you can predict it. The starter file murmur2.py contains Kafka's hash so you can check any key from the shell: python3 -c "from murmur2 import partition; print(partition('STORE-001', 6))"

To confirm placement on the cluster: kafka-get-offsets shows how many records each partition holds, and the console consumer with print.key and print.partition shows which key went where.

3.2 The Critical Distinction: Adding Partitions vs Keeping Order

⚠ Most Common Mistake When Scaling a Topic

Teams add partitions to get more consumer parallelism and assume nothing else changes. But the partition formula includes the partition count, so most keys start landing on a different partition.

REASON: Old records for a key stay in the old partition; new records go to the new one. A consumer can now read a key's newer events before its older ones — per-key ordering is broken across the change.

YOU MUST plan partition counts up front for keyed topics, or migrate to a new topic in a controlled way.

3.3 Producer Settings — Where the Trade-Off Lies

Setting	Higher / On Means	Cost	Measured By
acks=all (vs acks=1)	Write confirmed by all in-sync replicas — durable	Higher latency	perf-test avg / 99th latency
linger.ms + batch.size / compression.type	Bigger batches / fewer bytes sent	A few ms extra latency / CPU	perf-test records/sec and MB/sec

How to Decide Whether to Use a Key

Ask: "Must events for the same entity (order, store, customer) be processed in order?" Yes → use that entity's ID as the key. No → leave the key empty and let the sticky partitioner balance the load.

4. LAB TASKS

Figure: Lab 2 — Four-Task Producer Workflow: Unkeyed Publish → Keyed Publish → Partition Change → Producer Tuning
Figure: Expected Result — Keyed Placement Matches murmur2 Exactly, Key Movement After 6→8 Partitions Proven, Producer Settings Compared

TASK 1 — PUBLISH: Unkeyed Messages to clicks

1	**Publish and Read Unkeyed Messages** *Kafka Ops SOP — Day 4 Lab 2*	⏱ 5 min

You type six click events into the console producer, then read them back with their partition and offset.

⌨ Task 1 — Produce by typing, consume with partition and offset

```
# Step 1: start the producer, type these 6 lines (Enter after each), then press Ctrl+C
./k kafka-console-producer.sh --bootstrap-server $BS --topic clicks
page=/home user=U1
page=/shoes user=U2
page=/cart user=U1
page=/home user=U3
page=/checkout user=U1
page=/shoes user=U4

# Step 2: read everything back with partition and offset
./k kafka-console-consumer.sh --bootstrap-server $BS --topic clicks --from-beginning --max-messages 6 \
     --formatter-property print.partition=true --formatter-property print.offset=true

# Step 3: per-partition end offsets
./k kafka-get-offsets.sh --bootstrap-server $BS --topic clicks
```

Expected Result

```
Step 2:  6 lines such as  Partition:2   Offset:0   page=/home user=U1   then Processed a total of 6 messages
         Lines are grouped by partition — order is only guaranteed WITHIN a partition
```

```
Step 3:  clicks:0:a  clicks:1:b  clicks:2:c   with a + b + c = 6 (the split depends
on batching)
```

1.A Record the per-partition end offsets of clicks and check they add up to 6. Answer:
clicks:0:0
clicks:1:0
clicks:2:6
Total Sum: $0 + 0 + 6 = 6$ messages, which correctly matches the total number of unkeyed messages published.

1.B The consumer did not print the events in the order you typed them. Why is that allowed for unkeyed events? Answer:
Unkeyed events use Kafka's sticky partitioner, which groups messages into batches for a specific partition before rotating to another partition. Because there are no keys to enforce deterministic routing to specific partitions, ordering is only guaranteed *within* an individual partition, and the consumer reads across partitions without preserving the exact global chronological sequence in which they were typed.



TASK 2 — KEYS: Keyed Messages and Predictable Placement

2 **Publish 12 Keyed Orders and Verify Where Each Store Lands** ⏱ 7 min
Kafka Ops SOP — Day 4 Lab 2

First predict the partition of each store with Kafka's hash, then publish the file with parse.key enabled and check the result on the cluster.

⌨ Task 2 — Predict, publish, verify

```
# Step 1: predict (Kafka's default partitioner, 6 partitions)
python3 -c "from murmur2 import partition; print({f'STORE-00{i}': partition(f'STORE-00{i}', 6) for i in range(1, 7)})"
```

```
# Step 2: publish the keyed file (key and value separated by ':')
./k kafka-console-producer.sh --bootstrap-server $BS --topic orders \
    --property parse.key=true --property key.separator=: < data/keyed_orders.txt

# Step 3: per-partition counts
./k kafka-get-offsets.sh --bootstrap-server $BS --topic orders

# Step 4: read back with key and partition
./k kafka-console-consumer.sh --bootstrap-server $BS --topic orders --from-beginning
--max-messages 12 \
    --formatter-property print.key=true --formatter-property print.partition=true
```

Expected Output

```
Step 1:  {'STORE-001': 1, 'STORE-002': 5, 'STORE-003': 2, 'STORE-004': 4, 'STORE-
005': 5, 'STORE-006': 1}
Step 3:  orders:0:0  orders:1:4  orders:2:2  orders:3:0  orders:4:2  orders:5:4
Step 4:  every STORE-001 and STORE-006 line shows Partition:1; STORE-002/STORE-005
Partition:5; STORE-003 Partition:2;
         STORE-004 Partition:4.  (Kafka 4.2+ may print a note that --property is
deprecated — the command still works)
```

2.A Record the per-partition end offsets and confirm each matches the prediction from Step 1.
Answer:
Per-partition end offsets:

- orders:0:0
- orders:1:4
- orders:2:2
- orders:3:0
- orders:4:2
- orders:5:4

Confirmation: Yes, the offsets match the Step 1 murmur2 prediction perfectly:

- STORE-001 & STORE-006 land on **Partition 1** ($2 + 2 = 4$ messages).
- STORE-003 lands on **Partition 2** (2 messages).
- STORE-004 lands on **Partition 4** (2 messages).
- STORE-002 & STORE-005 land on **Partition 5** ($2 + 2 = 4$ messages).
- **Partitions 0 and 3** receive 0 messages.

2.B Six stores, six partitions — yet partitions 0 and 3 are empty and partitions 1 and 5 hold two stores each. Explain why. Answer:
Kafka's default partitioner uses a hashing algorithm ($\text{murmur2}(\text{key}) \pmod{\text{partitions}}$) to assign key placements, rather than a round-robin distribution. Because hashing aims for uniform distribution over a large dataset with thousands of distinct keys, small sample sizes (like 6 keys across 6 partitions)

naturally result in **hash collisions** (multiple store keys mapping to the exact same partition index) and leaving some partitions empty.

```
LabsKraft@ip-172-31-5-209:~/day4$ python3 -c "from murmur2 import partition; print({f'STORE-00{i}': partition(f'STORE-00{i}', 6) for i in ra
nge(1, 7)})"
{'STORE-001': 1, 'STORE-002': 5, 'STORE-003': 2, 'STORE-004': 4, 'STORE-005': 5, 'STORE-006': 1}
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-console-producer.sh --bootstrap-server $BS --topic orders \
     --property parse.key=true --property key.separator=: < data/keyed_orders.txt
Warning: --property is deprecated and will be removed in a future version. Use --reader-property instead.
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-get-offsets.sh --bootstrap-server $BS --topic orders
orders:0:0
orders:1:4
orders:2:2
orders:3:0
orders:4:2
orders:5:4
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-console-consumer.sh --bootstrap-server $BS --topic orders --from-beginning --max-messages 12 \
     --formatter-property print.key=true --formatter-property print.partition=true
The consumer rebalance protocol (KIP-848) is production-ready! Set group.protocol=consumer to try it out. See https://kafka.apache.org/docum
entation/#consumer_rebalance_protocol
Partition:1     STORE-001       order-0001,STORE-001,107.00
Partition:1     STORE-006       order-0006,STORE-006,142.00
Partition:1     STORE-001       order-0007,STORE-001,149.00
Partition:1     STORE-006       order-0012,STORE-006,184.00
Partition:2     STORE-003       order-0003,STORE-003,121.00
Partition:2     STORE-003       order-0009,STORE-003,163.00
Partition:4     STORE-004       order-0004,STORE-004,128.00
Partition:4     STORE-004       order-0010,STORE-004,170.00
Partition:5     STORE-002       order-0002,STORE-002,114.00
Partition:5     STORE-005       order-0005,STORE-005,135.00
Partition:5     STORE-002       order-0008,STORE-002,156.00
Partition:5     STORE-005       order-0011,STORE-005,177.00
Processed a total of 12 messages
LabsKraft@ip-172-31-5-209:~/day4$
```

⚠ STOP CHECK — Confirm Exact Placement, Not Assumption

Before moving to Task 3, confirm your orders offsets are exactly 0, 4, 2, 0, 2, 4 — if they differ, the file was published twice or without parse.key (see Section 6) and Task 3 will not match.

If in doubt — show your instructor your Step 3 output before proceeding to Task 3.

TASK 3 — STRATEGY: What Adding Partitions Does to Keys

3 **Increase orders to 8 Partitions and Re-Publish the Same Keys** ⏱ 6 min

Kafka Ops SOP — Day 4 Lab 2

Partitions can be added to a topic but never removed. You add two partitions and publish the same 12 orders again, then find where each store's new records went.

⌨ Task 3 — Alter, predict, re-publish, compare

```
# Step 1: increase partitions 6 → 8
./k kafka-topics.sh --bootstrap-server $BS --alter --topic orders --partitions 8

# Step 2: predict the new placement
```

```
python3 -c "from murmur2 import partition; print({f'STORE-00{i}': partition(f'STORE-00{i}', 8) for i in range(1, 7)})"
```

```
# Step 3: publish the same file again and check the counts
./k kafka-console-producer.sh --bootstrap-server $BS --topic orders \
    --property parse.key=true --property key.separator=: < data/keyed_orders.txt
./k kafka-get-offsets.sh --bootstrap-server $BS --topic orders
```

Expected Output

```
Step 2:  {'STORE-001': 7, 'STORE-002': 1, 'STORE-003': 0, 'STORE-004': 6, 'STORE-005': 5, 'STORE-006': 5}
Step 3:  orders:0:2  orders:1:6  orders:2:2  orders:3:0  orders:4:2  orders:5:8  orders:6:2  orders:7:2  (total 24)
STORE-001 now has 2 old records in partition 1 and 2 new records in partition 7
```

3.A For STORE-001 and STORE-004, record the partition before and after the change. Answer:

STORE-001:

- **Before (6 partitions):** Partition **1**
- **After (8 partitions):** Partition **7**

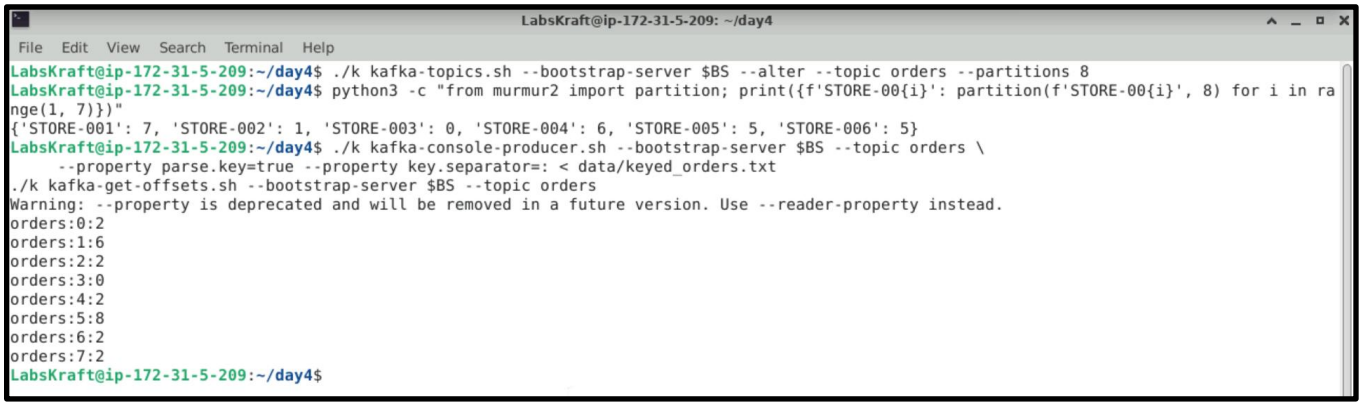
STORE-004:

- **Before (6 partitions):** Partition **4**
- **After (8 partitions):** Partition **6**

3.B A consumer group reads partition 7 faster than partition 1. What can happen to STORE-001's events, and how should the team have scaled instead? Answer:

What can happen: Newer events for STORE-001 (written to partition 7 after the change) can be processed *before* older events (which still sit in partition 1), resulting in **out-of-order event processing** and breaking business logic for that store.

How the team should have scaled: The team should have planned the required partition count up front for keyed topics, or performed a controlled migration by creating a new topic with 8 partitions and routing traffic after existing partition queues were completely drained.



TASK 4 — TUNE: Producer Configuration and Throughput

4	**Compare acks, Batching and Compression With the Perf Test** *Kafka Ops SOP — Day 4 Lab 2*	⏱ 6–7 min

kafka-producer-perf-test sends synthetic records as fast as possible and reports throughput and latency. You run the same 200,000-record test three times with different producer settings.

⌨ Task 4 — Three producer configurations on perf-test

```
# Run A: acks=1 (leader only)
./k kafka-producer-perf-test.sh --bootstrap-server $BS --topic perf-test --num-records 200000 --record-size 200 \
    --throughput -1 --command-property acks=1

# Run B: acks=all (all in-sync replicas)
./k kafka-producer-perf-test.sh --bootstrap-server $BS --topic perf-test --num-records 200000 --record-size 200 \
    --throughput -1 --command-property acks=all

# Run C: acks=all + bigger batches + lz4 compression
./k kafka-producer-perf-test.sh --bootstrap-server $BS --topic perf-test --num-records 200000 --record-size 200 \
    --throughput -1 --command-property acks=all --command-property linger.ms=20 \
    --command-property batch.size=131072 --command-property compression.type=lz4
```

⌨ Expected Output

```
Each run ends with one summary line, for example:

200000 records sent, 85000.0 records/sec (16.21 MB/sec), 45.2 ms avg latency, 400.0 ms max latency, ... 99th ...

Pattern (exact numbers depend on your VM): B has higher latency than A; C has the highest records/sec.
```

4.A Record records/sec, MB/sec, avg latency and 99th percentile latency for runs A, B and C. Answer:
Run A (acks=1):

- **Records/sec:** 19986.01
- **MB/sec:** 3.81 MB/sec
- **Avg Latency:** 64.63 ms
- **99th Percentile Latency:** 333 ms

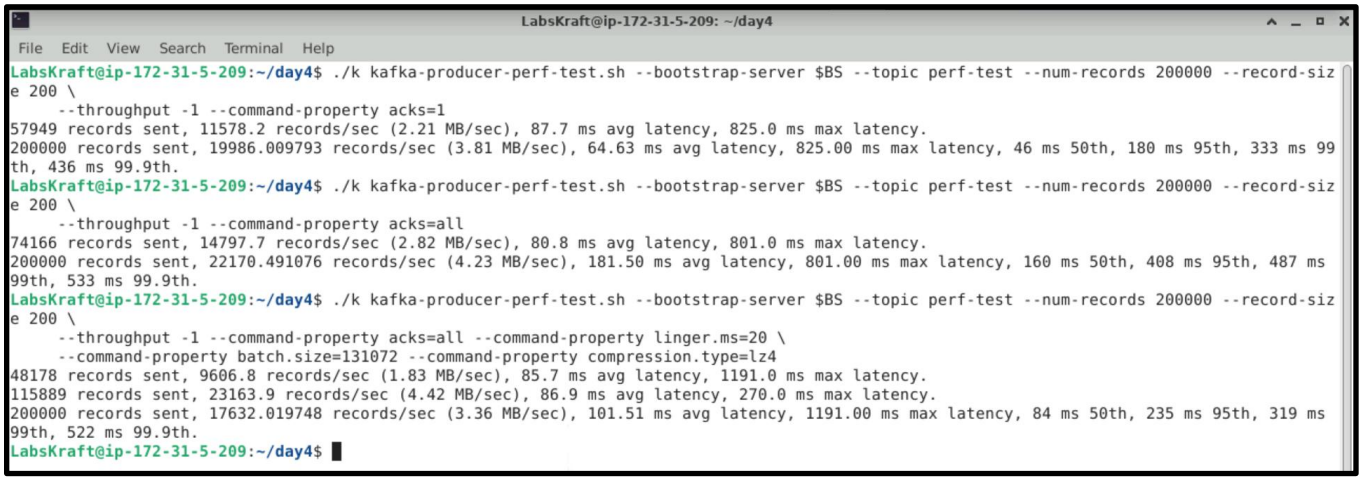
Run B (acks=all):

- **Records/sec:** 22170.49
- **MB/sec:** 4.23 MB/sec
- **Avg Latency:** 181.50 ms
- **99th Percentile Latency:** 487 ms

Run C (acks=all + linger.ms=20 + batch.size=131072 + compression.type=lz4):

- **Records/sec:** 17632.02
- **MB/sec:** 3.36 MB/sec
- **Avg Latency:** 101.51 ms
- **99th Percentile Latency:** 319 ms

4.B Which configuration would you use for payments, and which for clickstream? Justify with your numbers. Answer:
For Payments: Use **Configuration B (or C)** with acks=all. Payments require zero data loss durability guarantees. acks=all ensures every in-sync replica acknowledges the write before confirming. Even though average latency increases to **181.50 ms**, durability takes priority over speed for financial transactions.
For Clickstream: Use **Configuration C (or A)**. Clickstream logs generate massive volumes where high throughput, batch efficiency, and low tail latency are essential. Configuration C achieves the lowest 99th percentile tail latency (**319 ms**) and significantly compresses data on the wire with LZ4 compression, saving CPU and bandwidth.



5. VALIDATION — CONFIRM FULL READINESS

Do not close the incident or call for instructor sign-off until ALL 8 validation steps below are completed and recorded.
Figure: Lab 2 — Producers & Partitioning Validation Checklist

5.1 Validation Task Set

Validation Check 1 — Unkeyed messages stored

Validation 1

```
./k kafka-get-offsets.sh --bootstrap-server $BS --topic clicks

# PASS criterion: end offsets add up to 6
```

V1: Do the clicks partitions hold 6 messages in total? Answer:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-get-offsets.sh --bootstrap-server $BS --topic clicks
clicks:0:0
clicks:1:0
clicks:2:6
LabsKraft@ip-172-31-5-209:~/day4$
```

Validation Check 2 — orders now has 8 partitions

Validation 2

```
./k kafka-topics.sh --bootstrap-server $BS --describe --topic orders | head -1

# PASS criterion: PartitionCount: 8
```

V2: Does orders have 8 partitions? Answer:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-topics.sh --bootstrap-server $BS --describe --topic orders | head -1
Topic: orders   TopicId: 6jk11ykdTr-hyhwQT1EwdQ PartitionCount: 8       ReplicationFactor: 3    Configs: min.insync.replicas=1
LabsKraft@ip-172-31-5-209:~/day4$
```

Validation Check 3 — Exact keyed placement

Validation 3

```
./k kafka-get-offsets.sh --bootstrap-server $BS --topic orders

# PASS criterion: 0:2 1:6 2:2 3:0 4:2 5:8 6:2 7:2
```

V3: Do the orders offsets match exactly? Answer:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-get-offsets.sh --bootstrap-server $BS --topic orders
orders:0:2
orders:1:6
orders:2:2
orders:3:0
orders:4:2
orders:5:8
orders:6:2
orders:7:2
LabsKraft@ip-172-31-5-209:~/day4$
```

Validation Check 4 — Prediction matches the cluster

Validation 4

```
python3 -c "from murmur2 import partition; print(partition('STORE-004', 6),
partition('STORE-004', 8))"

# PASS criterion: 4 6 - matching where STORE-004's records were found
```

V4: Does the hash prediction match the cluster for STORE-004? Answer:

```
LabsKraft@ip-172-31-5-209:~/day4$ python3 -c "from murmur2 import partition; print(partition('STORE-004', 6), partition('STORE-004', 8))"
4 6
LabsKraft@ip-172-31-5-209:~/day4$
```

Validation Check 5 — Perf-test data written

Validation 5

```
./k kafka-get-offsets.sh --bootstrap-server $BS --topic perf-test | awk -F: '{s+=$3}
END {print s}'

# PASS criterion: 600000 (3 runs × 200,000)
```

V5: Does perf-test hold 600,000 records? Answer:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-get-offsets.sh --bootstrap-server $BS --topic perf-test | awk -F: '{s+=$3} END {print s}'
600000
LabsKraft@ip-172-31-5-209:~/day4$
```

Validation Check 6 — Producer results recorded

Validation 6

```
# Answer box 4.A

# PASS criterion: throughput and latency for runs A, B and C
```

V6: Are all three perf-test results recorded? Answer:
Yes, all three kafka-producer-perf-test runs have been recorded with their complete throughput and latency metrics:

- **Run A (acks=1):** 19,986.01 rec/sec (3.81 MB/sec), **Avg Latency:** 64.63 ms, **99th:** 333 ms
- **Run B (acks=all):** 22,170.49 rec/sec (4.23 MB/sec), **Avg Latency:** 181.50 ms, **99th:** 487 ms
- **Run C (acks=all + batching & LZ4):** 17,632.02 rec/sec (3.36 MB/sec), **Avg Latency:** 101.51 ms, **99th:** 319 ms

Validation Check 7 — Replicas healthy after the load

Validation 7

```
./k kafka-topics.sh --bootstrap-server $BS --describe --under-replicated-partitions

# PASS criterion: no output
```

V7: Are all replicas in sync after the tests? Answer:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-topics.sh --bootstrap-server $BS --describe --under-replicated-partitions
LabsKraft@ip-172-31-5-209:~/day4$
```

Validation Check 8 — Full lab record complete

⌨ Validation 8 — Final completeness check

```
# Confirm every answer box (PRE-LAB, 1.A through 4.B) is completed

# PASS criterion: every answer box holds real output (offsets, partitions,
throughput)
```

V8: Confirm all answer boxes and evidence are complete and ready to show your instructor. Answer: _____

✅ STOP HERE — Show your completed validation record to your instructor before proceeding to Section 7.

General - RPS Data

6. TROUBLESHOOTING — COMMON ISSUES & FIXES

Issue / Error	Cause	Diagnosis	Fix
orders offsets are double the expected values	keyed_orders.txt was published twice in Task 2	kafka-get-offsets totals 24 before Task 3	Ask the instructor to recreate orders (delete + create 6 partitions) and repeat Task 2
All 12 orders spread unevenly with no STORE pattern	parse.key=true was missing — the whole line became the value	Consumer with print.key=true shows null keys	Recreate orders and publish again with both --property options
Console consumer never exits	Fewer messages than --max-messages exist	kafka-get-offsets totals	Press Ctrl+C, or add --timeout-ms 10000
Unrecognized option --command-property / --formatter-property	Image older than Kafka 4.2	docker image ls apache/kafka	Use the 4.3.1 image, or the older names --producer-property / --property
Perf test very slow or errors	VM overloaded	docker stats shows high CPU	Lower --num-records to 50000 and note it in 4.A

General - RPS Data

7. INSTRUCTOR DEBRIEF — 8 MINUTES

7.1 Discussion Questions

Discussion 1 — Partition Strategy

Without looking at notes: state the rule for keyed records and for unkeyed records.

Why can two keys share a partition while another partition is empty?

Key answer to explore: hashing is uniform over MANY keys; with few keys, collisions and empty partitions are normal.

Discussion 2 — Changing Partition Counts

Which stores moved partitions in Task 3, and what does that mean for a consumer that relies on per-store order?

What alternatives exist to increasing partitions on a keyed topic?

Key insight: plan partitions for future parallelism, or migrate to a new topic with a cut-over.

Discussion 3 — Producer Settings

Compare the class's results for A, B and C. Why does batching increase throughput?

Why is acks=all slower, and when is that worth it?

Key answer: fewer, larger requests amortise overhead; acks=all waits for followers but survives a leader failure.

Discussion 4 — Production Considerations

Which producer defaults would you change for a high-volume clickstream topic, and which must you never weaken for payments?

Answer: clickstream → larger linger/batch + compression; payments → keep acks=all with idempotence enabled (Lab 3).

7.2 Common Mistakes — Learn From These

General - RPS Data

Mistake	Why It Happens	Consequence	Prevention
Adding partitions to a keyed topic	Need more consumers quickly	Per-key ordering breaks	Plan partitions up front
Assuming N keys fill N partitions	Thinking hashing is round-robin	Idle and overloaded partitions	Check placement; use many distinct keys
No key when order matters	Simpler producer code	Events of one entity processed out of order	Key by the entity ID
acks=1 for critical data	Faster benchmarks	Data loss if the leader fails before replication	acks=all for critical topics
Tuning without measuring	Copying settings from articles	Unknown effect on latency	Use kafka-producer-perf-test

General - RPS Data

8. EXTENSION TASKS (Advanced Learners)

Extension A — Predict a Hot Key

Use murmur2.py to find which partition STORE-099 would use on 6 and on 8 partitions.

Publish one record with that key and confirm with kafka-get-offsets.

Key question: how could a single very busy store overload one partition? (Lab 5)

Extension B — Message Size Limits

Run the perf test with --record-size 2000000 and observe the error.

Find the producer setting max.request.size and the topic setting max.message.bytes.

Research: why are very large Kafka messages usually stored elsewhere (e.g. object storage) with only a reference in Kafka?

Extension C — Compression Ratio

Repeat Run C with compression.type=zstd and with gzip.

Compare MB/sec and use kafka-log-dirs.sh to compare the bytes stored on disk.

Explain the CPU-versus-network trade-off of each codec.

General - RPS Data

9. LAB COMPLETION & SIGN-OFF

9.1 Submission Checklist

✓	Completion Criterion	Status
☐	Unkeyed messages published and read back with partition and offset	☐ Pass / ☐ Fail
☐	Keyed orders published; placement predicted with murmur2 and matched exactly (0,4,2,0,2,4)	☐ Pass / ☐ Fail
☐	Partition increase 6→8 performed; key movement proven (0,6,2,0,2,8,2,2)	☐ Pass / ☐ Fail
☐	Three producer configurations measured and compared	☐ Pass / ☐ Fail
☐	All 8 validation checks completed and recorded	☐ Pass / ☐ Fail
☐	All observation/answer boxes in this guide completed	☐ Pass / ☐ Fail

9.2 Grading Criteria

Assessment Area	Criteria	Marks	Weight
Task 1 — Unkeyed Publish	Messages published and read back; ordering explained	15	15%
Task 2 — Keyed Publish	Placement predicted and verified exactly; collisions explained	30	30%
Task 3 — Partition Change	Key movement evidenced; ordering impact explained	25	25%
Task 4 — Producer Tuning	Three runs measured; configuration choices justified	20	20%
Validation & Documentation	All 8 validation checks completed; answers complete and accurate	10	10%
TOTAL		100	100%

9.3 Instructor Sign-Off

General - RPS Data

Field	Value
Lab Result	☐ PASS ☐ PARTIAL ☐ REPEAT REQUIRED | Score: _____ / 100
Instructor Name	_____
Instructor Signature & Date	_____
Remarks / Feedback	_____ _____
Completion Date & Time	_____

General - RPS Data

10. QUICK REFERENCE — KAFKA PRODUCER COMMAND CARD

Bookmark this page. Every command you need in SOP order.

Category	Command	Purpose
Produce	./k kafka-console-producer.sh --bootstrap-server $BS --topic t	Type messages (Ctrl+C to stop)
Produce	... --property parse.key=true --property key.separator=: < file	Publish keyed lines from a file
Produce	... --command-property acks=all	Pass producer settings (Kafka 4.2+ name)
Consume	./k kafka-console-consumer.sh --bootstrap-server $BS --topic t --from-beginning --max-messages n	Read n messages
Consume	--formatter-property print.key=true / print.partition=true / print.offset=true	Show key, partition, offset
Offsets	./k kafka-get-offsets.sh --bootstrap-server $BS --topic t	End offset per partition
Predict	python3 -c "from murmur2 import partition; print(partition('KEY', n))"	Partition of a key
Topics	./k kafka-topics.sh --bootstrap-server $BS --alter --topic t --partitions n	Increase partitions
Perf	./k kafka-producer-perf-test.sh --bootstrap-server $BS --topic t --num-records n --record-size b --throughput -1	Throughput and latency test